

Disclaimer: Prosjektet er laget selv, gjennom bruken av erfaring og google, der AI ble mest brukt for å finne feil i logikken. Prosjektet ble gjennomført i en periode på 14 dager.

I dette prosjektet har jeg laget en java program som kan liste opp spill i databasen kalt for “gameStop”. Grunnen til at jeg lager dette programmet er fordi jeg går gjennom Java på nytt for å refreshe kunnskapen og jeg vil bruke det jeg har lært til å lage et prosjekt som interesserer meg. Databasen ble laget manuelt i mySQL programmet som en start for å sette opp alle nødvendige deler av programmet, slik at Java-programmet kan vise info fra databasen, samtidig som man kan utføre CRUD-operasjoner når prosjektet er ferdiglagt.

Fremgangsmåte:

Første fase: Oppbygging av prosjektets funksjonaliteter

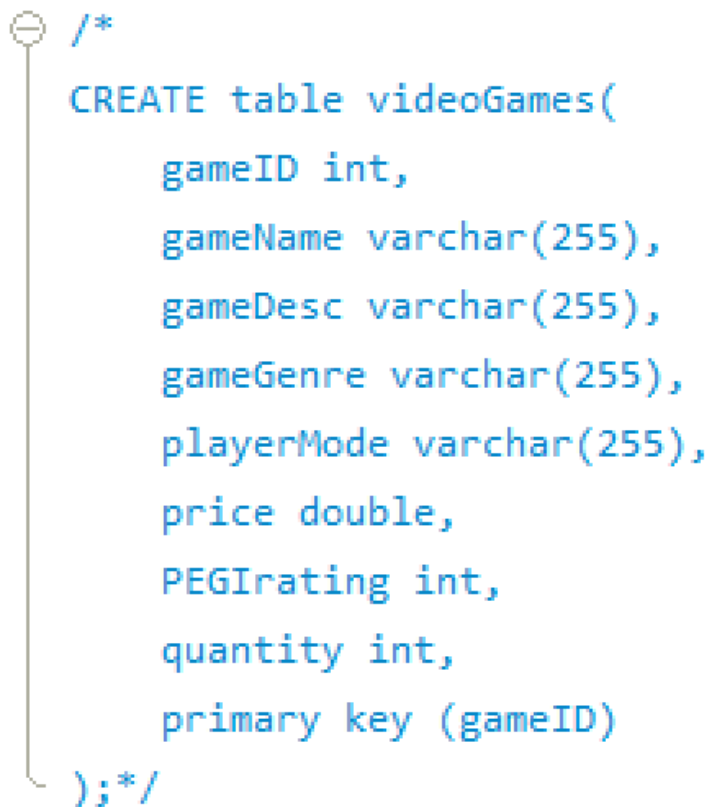
Lagde databasen “gameStop”:

```
1      /*CREATE DATABASE gameStop;*/
```

Figurbilde 1: Lager databasen gameStop

Lagde tabellene “videoGames”, “customer” og “orders, hver med deres nødvendige data:

tabellen “videoGames”:



```
/*  
CREATE table videoGames(  
    gameID int,  
    gameName varchar(255),  
    gameDesc varchar(255),  
    gameGenre varchar(255),  
    playerMode varchar(255),  
    price double,  
    PEGIrating int,  
    quantity int,  
    primary key (gameID)  
);*/
```

Figurbilde 2: Lager tabellen “videoGames” med de ovennevnte kolonnene

tabellen "customer":

```
/*  
CREATE table customer(  
    customerID int,  
    customerFirstName varchar(255),  
    customerLastName varchar(255),  
    customerEmail varchar(255),  
    primary key (customerID)  
);
```

Figurbilde 3: Lager tabellen "customer" med de ovennevnte kolonnene

tabellen "orders":

```
⊖ CREATE TABLE orders(  
    orderID int,  
    customerFristName varchar(255),  
    customerLastName varchar(255),  
    gameName varchar(255),  
    gameID int,  
    purchaseData varchar(255),  
    returned boolean,  
    primary key (orderID)  
);
```

Figurbilde 4: Lager tabellen "orders" med de ovennevnte kolonnene

La inn test data i “videogames”, “customer” og “orders” slik at jeg kan få det printet ut via terminalen i et senere tidspunkt:

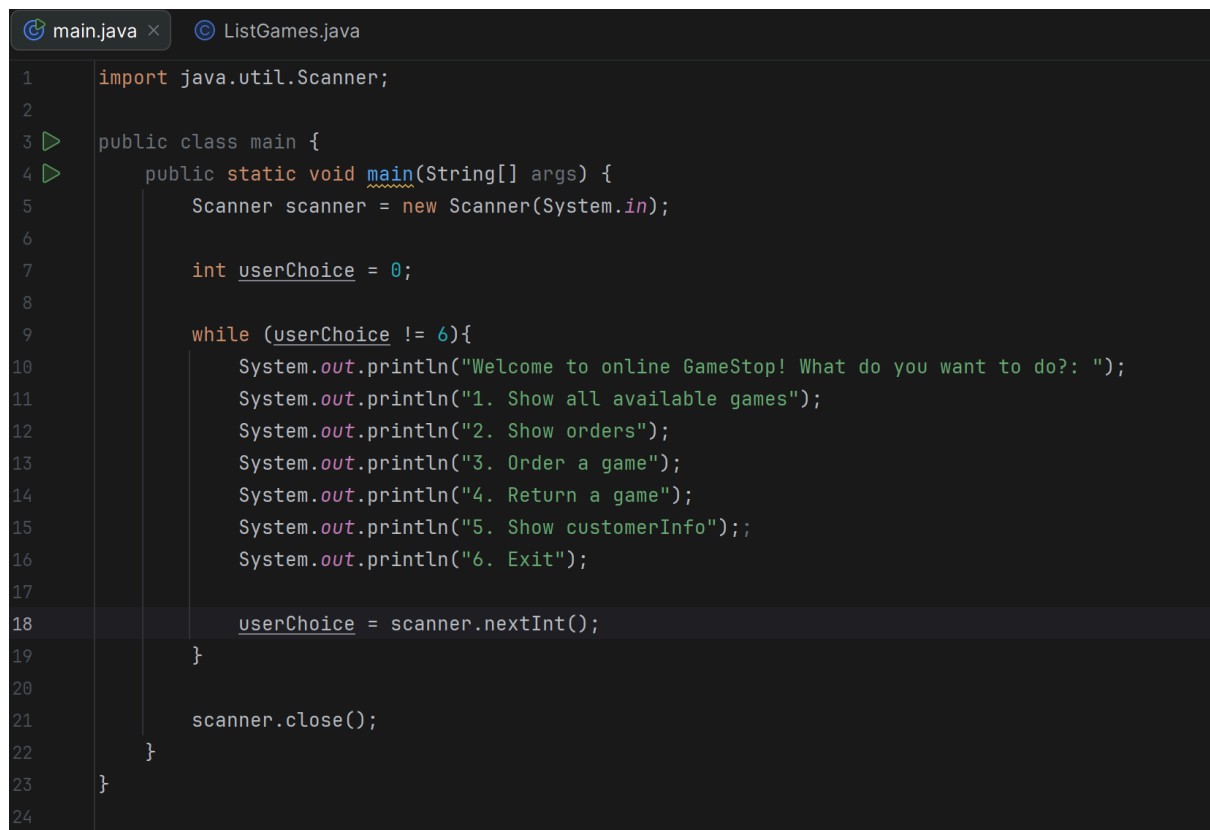
```
/*
INSERT INTO videogames(gameID, gameName, gameDesc, gameGenre, playerMode, price, PEGIrating, quantity)
VALUES(1,"God Of War: Ragnarok", "Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go",

/*
INSERT INTO customer(customerID, customerFirstName, customerLastName, customerEmail)
VALUES(1, "John", "Halo", "john2026@halo.com")*/

INSERT INTO orders(orderID, customerFristName, customerLastName, gameName, gameID, purchaseData, returned)
VALUES (1344, "John", "Halo", "God Of War: Ragnarok", 1, "28-02-2026", false)
```

Skjerm bilde 5: Insertion av data til tabellene “videogames”, “customer” og “orders”

Lagde en “main” klasse, hvor man foreløpig har en scanner som tillater brukeren å velge 6 forskjellige alternativer: “1. Show all available games”, “2. Show orders”, “3. Order a game”, “4. Return a game”, “5. Show customerinfo” og “6. exit”



```
main.java x ListGames.java
1 import java.util.Scanner;
2
3 public class main {
4     public static void main(String[] args) {
5         Scanner scanner = new Scanner(System.in);
6
7         int userChoice = 0;
8
9         while (userChoice != 6){
10             System.out.println("Welcome to online GameStop! What do you want to do?: ");
11             System.out.println("1. Show all available games");
12             System.out.println("2. Show orders");
13             System.out.println("3. Order a game");
14             System.out.println("4. Return a game");
15             System.out.println("5. Show customerInfo");
16             System.out.println("6. Exit");
17
18             userChoice = scanner.nextInt();
19         }
20
21         scanner.close();
22     }
23 }
24
```

Figurbilde 6: Første utkast av menyen til prosjektet

For å kunne teste om alternativ 1 fungerer, så lagde jeg en “ListGames” klasse, som vist i skjermbildet under. Feltene er tildelt med hardkoda verdier for å teste om jeg kan få skrevet det ut på terminalen. Dette ble fikset på senere i rapporten.

```
public class ListGames { 2 usages
    void showGames(){ 1 usage
        int gameId = 1;
        String gameName = "God Of War: Ragnarok";
        String gameDescription = "Something";
        String gameGenre = "Action-adventure";
        String playerMode = "Single-Player";
        double price = 899;
        int PEGIrating = 18;
        int quantity= 3;

        System.out.println(gameID);
        System.out.println(gameName);
        System.out.println(gameDescription);
        System.out.println(gameGenre);
        System.out.println(playerMode);
        System.out.println(price);
        System.out.println(PEGIrating);
        System.out.println(quantity);
    }
}
```

Figurbilde 7: Feltene med tildelte verdier i metoden "showGames" i klassen "ListGames"

Dette er resultatet når brukeren trykker på valg nummer 1. Menyen var bugga, der valgene kom frem igjen før resultatet fra "showGames" metoden kom frem:

```
6. EXIT
1
Welcome to online GameStop! What do you want to do?:
1. Show all available games
1
God Of War: Ragnarok
Something
Action-adventure
Single-Player
899.0
18
3
```

Figurbilde 8: Resultatet som kommer frem når brukeren velger valg 1

Oppdaterte koden i “main”, der jeg erstattet “if-else”-delen av koden med switch case. Dette gjør dette lettere å få mer oversikt, gjør koden mer lesbar og jeg kan lettere ekspandere på den dersom det er behov. Jeg lagde også objekt instanser av “listGames”, slik at jeg kan skrive inn verdier gjennom parameterne i “showGames” metoden. Endringen fjernet også buggen hvor menyen kom frem først før resultatene fra valgene (1-5) ble fremvist, som forklart tidligere i figurbilde 8:

```
switch ((userChoice = scanner.nextInt())) {
    case 1:
        listGames.showGames( gameId: 1, gameIdName: "God of War: Ragnarok", gameIdDescription: "Dad of War",
            gameIdGenre: "Action-Adventure", gameIdPlayerMode: "Single-player", gameIdPrice: 899, gameIdPEGRating: 18, gameIdQuantity: 3);
        listGames1.showGames( gameId: 2, gameIdName: "Ghost of Yotei", gameIdDescription: "Twin wolves",
            gameIdGenre: "Action-adventure", gameIdPlayerMode: "Single-player", gameIdPrice: 699, gameIdPEGRating: 18, gameIdQuantity: 5);
        break;
    case 2:
        System.out.println("Here are the current orders: ");
        break;
    case 3:
        System.out.println("What games do you want to order?: ");
        break;
    case 4:
        System.out.println("What games do you want to return?: ");
        break;
    case 5:
        System.out.println("Here is customer info: ");
        break;
    case 6:
        System.out.println("Thank you for visiting online GameStop!");
    default:
        System.out.println("Invalid choice.");
        break;
}
```

Figurbilde 9: Menyene i main, der jeg har erstattet if-else med switch-case. Case 1 bruker også en instans av “listGames” for å liste opp spill dynamisk.

Gjorde alle attributtene i “ListGames” klassen til parametere, da det tillater meg å gjenbruke metoden hver gang jeg lager nye instanserte objekter fra klassen, der hver instantierte objekt er unike i forhold til hverandre, som vist i figurbilde 9 med “listgames1” og “listgames” i case 1:

```
public class ListGames { 4 usages
    void showGames(int gameID, String gameName, String gameDescription, String gameGenre, String playerMode, 2 usages
        double price, int PEGIrating, int quantity){

        System.out.println("ID: " + gameID + " Game Name: " + gameName + " Game Description: " + gameDescription);
        System.out.println("Player Mode: " + playerMode);
        System.out.println("Price: " + price);
        System.out.println("PEGIrating: " + PEGIrating);
        System.out.println("Quantity: " + quantity);
    }
}
```

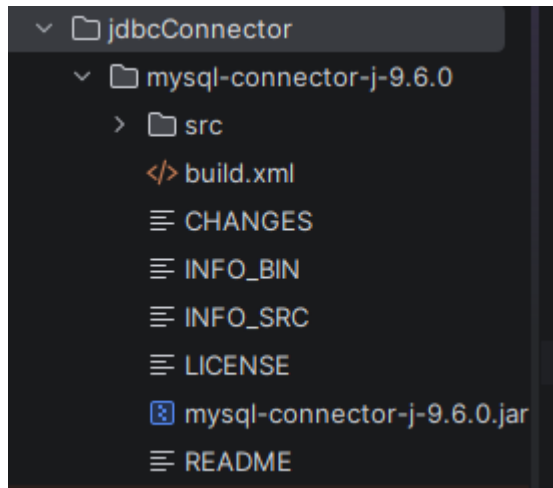
Figurbilde 10: “showGames” metoden, der jeg har flyttet variablene inn i parantesen og gjort dem om til parametere til metoden

Lagde klassen “DatabaseConnection”, der den skal håndtere databasekobling mellom mysql og java programmet:

```
main.java DatabaseConnection.java ListGames.java
1 public class DatabaseConnection {
2
3     public static void main(String[] args) {
4         try{
5             Class.forName("com.mysql.cj.jdbc.Driver").getDeclaredConstructor().newInstance();
6         }catch(Exception e){
7             System.out.println("Error: Couldn't load JDBC driver");
8         }
9     }
10 }
11 |
```

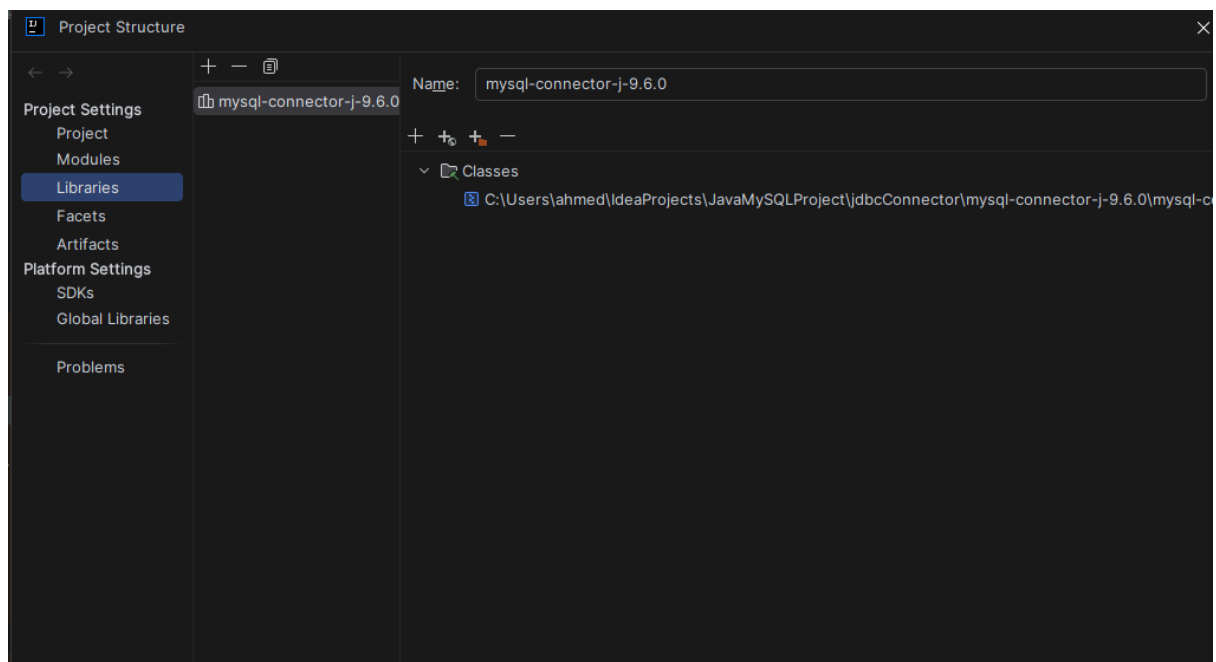
Figurbilde 11: Oppsett av “DatabaseConnection” klassen

Lastet ned “mysql-connector” mappen (inkludert .jar-filen) fra mysql sine nettsider på maskinen og importerte den inn i mappen “jdbcConnector”:



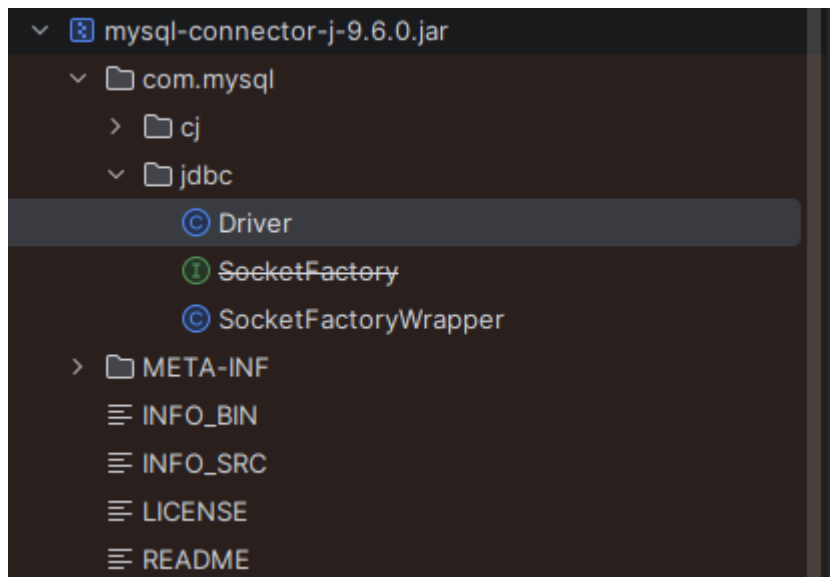
Figurbilde 12: Mysql-connector mappen, importert til prosjektet.

Etter å søkt og testet med jdbc-koblingen i 3 timer fant jeg ut at jeg måtte legge .jar filen inn i "Libraries"-delen av prosjektet, slik at jeg kan ha tilgang til filen i koden. Den gamle måte ved å bruke "Class.forName()" og deretter skrive inn filstien til .jar-filen funket ikke, da det var utdattert:

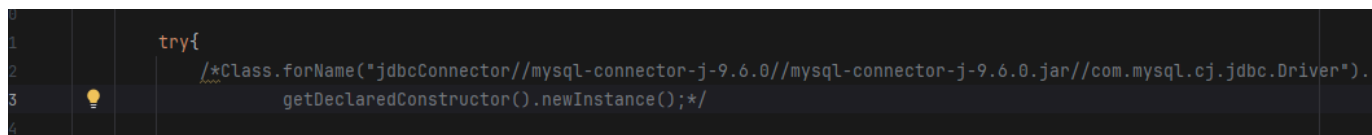


Figurbilde 13: Importerte .jar-filen til "Libraries" gjennom "project structure" innstillingene

Dette gjorde at kobling “[com.mysql.jdbc.*](#)” mappen (og mer) ble importert automatisk:



Figurbilde 14: De nødvendige filene og mappene som trengs for å danne databasekoblingen



Figurbilde 15: Kommenterte ut “Class.forName” delen av koden da det var depreceated i versjon 8.0 mysql.

Fikk dette opp som resultat:



Figurbilde 16: Bekreftelse på at kobling til mysql databasen “gameStop” og java fungerte.

Har fjernet vekk “main” metoden i “Databaseconnection” klassen og laget en “selectCustomer” metode, som vist i figurbilde 17. Da slipper jeg å ha to main metoder, hvor jeg kan heller kalle metoden når brukerne trykker på valg 5 i “main klassen”. Markerte over “password” og “username” feltene for sikkerhetsgrunner:


```
public class DatabaseConnection { 2 usages
    void selectCustomer() { 1 usage
        Connection connection = null;

        try{
            connection = DriverManager.getConnection( url: "jdbc:mysql://localhost:3306/gamestop", " ", " ");

            Statement statement = connection.createStatement();

            ResultSet resultSet = statement.executeQuery( sql: "SELECT * FROM customer");

            while (resultSet.next()) {
                System.out.println(resultSet.getString( columnLabel: "customerFirstName") + " * " + resultSet.getString( columnLabel: "customerLastName"));
            }

            System.out.println("Connection successful");

            resultSet.close();
            statement.close();
            connection.close();
        } catch (Exception e) {
            System.out.println("Connection failed");
            e.printStackTrace();
        }
    }
}
```

Figurbilde 17: Koden for “selectCustomer”, der den lister opp all customers i tabellen “customer”

Til å begynne med brukte jeg “Statement” for å kunne utføre sql spørringene, da det fungerte godt nok i begynnelsen, ettersom jeg ikke hadde implementert brukerinput gjennom scanner objektet, hvor inputet kunne bli registrert i databasen. Dette ble håndtert senere i rapporten.

Lager en instanse av “DatabaseConnection” i main:

```
DatabaseConnection databaseConnection = new DatabaseConnection();
```

Figurbilde 18: Instanse av klassen “DatabaseConnection”, kalt for “databaseConnection”

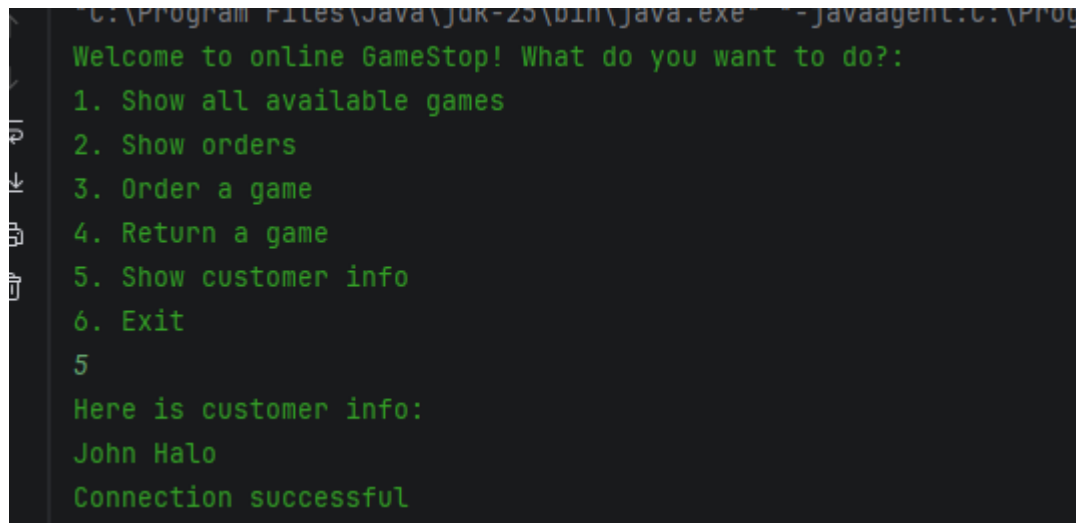
Plasserte instansen og kaller på metoden til “DatabaseConnection” klassen i case 5:

```
case 5:
    System.out.println("Here is customer info: ");

    databaseConnection.selectCustomer();
```

Figurbilde 19: Kaller på instansen av klassen og metoden i case 5

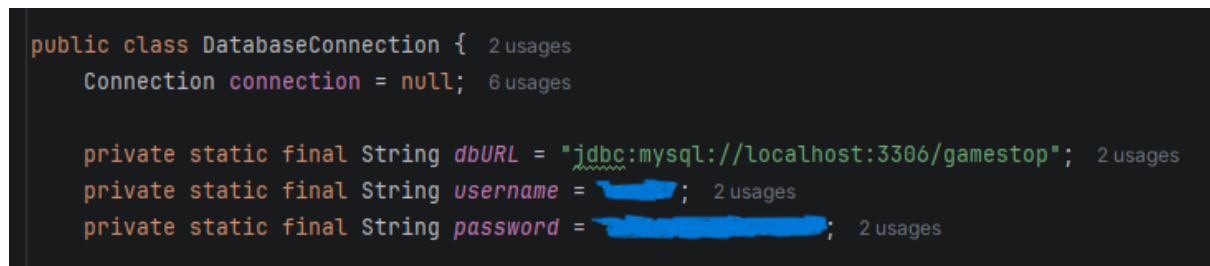
Får dette som resultat, der man får opp customer info fra “customer” tabellen i databasen:



```
C:\Program Files\Java\jdk-25\bin\java.exe -javaagent:C:\Prog...
Welcome to online GameStop! What do you want to do?:
1. Show all available games
2. Show orders
3. Order a game
4. Return a game
5. Show customer info
6. Exit
5
Here is customer info:
John Halo
Connection successful
```

Figurbilde 20: John Halo blir printet ut i terminalen når brukeren velger valg 5

Modifiserte koden i "DatabaseConnection" klassen, der jeg har gjort at database-urlen, brukernavnet og passordet er plassert i egne felt, kalt for “dbURL”, “username” og “password”. Dette gjør det mulig å bruke connection om igjen for de andre tabellene og deretter utføre CRUD operasjoner på et senere tidspunkt. Tusjet over passordet og brukernavnet for sikkerhetsgrunner:



```
public class DatabaseConnection { 2 usages
    Connection connection = null; 6 usages

    private static final String dbURL = "jdbc:mysql://localhost:3306/gamestop"; 2 usages
    private static final String username = " "; 2 usages
    private static final String password = " "; 2 usages
```

Figurbilde 21: Plasserte koblingen til databasen, passord og brukernavn i egne felt. Gjør det lettere når jeg lager de andre metodene for å bruke feltene i stedet for å skrive kredensialene for hver enkelte metode/CRUD-operasjon hver gang.

Deretter lagde jeg en metode kalt for “selectGame”, der brukeren kan printe ut verdien i tabellen “videogames”, når de trykker på valg 1:

```

void selectGame(){ 1 usage
try{
    connection = DriverManager.getConnection(dbURL, username, password);

    Statement statement = connection.createStatement();

    ResultSet resultSet = statement.executeQuery( sql: "SELECT * FROM videoGames");

    while (resultSet.next()){
        System.out.println(resultSet.getString( columnLabel: "gameName") + " " + resultSet.getString( columnLabel: "gameDesc")
            + " " + resultSet.getString( columnLabel: "gameGenre") + " " + resultSet.getString( columnLabel: "playerMode"));
    }

    resultSet.close();
    statement.close();
    connection.close();
}catch (Exception e){
    System.out.println("Failed to load data from table 'videoGames' ");
    e.printStackTrace();
}
}

```

Figurbilde 22: Koden for “selectGame” metoden, der den lister opp alle spill i “videoGames” tabellen.

```

switch ((userChoice = scanner.nextInt()))
    case 1:
        databaseConnection.selectGame();
        break;

```

Figurbilde 23: Plasserte “selectGame” metoden fra “DatabaseConnection” klassen i case 1

Dette er resultatet når valg 1 er valgt:

```

1
God Of War: Ragnarok Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go Action-adventure Single-Player

```

Figurbilde 24: Utprinting av første rad i tabellen “videogames”.

Formattering er ikke helt ryddig og fin å se på, men det ble håndtert senere i rapporten.

Lagde mer testdata i MySQL for å ha en seleksjon av spill tilgjengelig for brukeren:

```

--INSERT INTO videogames(gameID, gameName, gameDesc, gameGenre, playerMode, price, PEGIrating, quantity)
VALUES
(3, "Destiny 2", "Dive into the world of Destiny 2 to explore the mysteries of the solar system and experience responsive first-person shooter combat. Unlock powerful elemental abilities and collect unique gear to customize you
"First-Person Shooter", "Multiplayer", 0, 16, 50)
/(2, "Ghost Of Yotei", "AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES", "Action-adventure", "Single-Player", 699, 18, 6)/

```

Figurbilde 25: La til testdata for spillene “Ghost Of Yotei” og “Destiny 2” i MySQL

Bekreftet resultatet via java når jeg kjørte option 1 på nytt:

```

Here is all currently available games:
God Of War: Ragnarok
Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go
Action-adventure
Single-Player
Ghost Of Yotei
AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES
Action-adventure
Single-Player
Destiny 2
Dive into the world of Destiny 2 to explore the mysteries of the solar system and experience responsive first-person shooter combat. Unlock powerful elemental abilities and collect unique gear to customize your Guardian's
First-Person Shooter
Multiplayer

```

Figurbilde 26: Alle spillene i tabellen “videogames”.

Lagde deretter en ny metode i “DatabaseConnection” kalt for “createGame”. Her fant jeg ut gjennom google at jeg måtte bruke “.executeUpdate” og ikke “.resultSet” for å inserte data til

tabellen “videoGames”. Midlertidig er det hardkoda verdier for å se om metoden fungerer når den blir aktivert i main, men det ble fikset på senere i rapporten:

```
void createGame(){ 1 usage
    try{
        connection = DriverManager.getConnection(dbURL, username, password);
        Statement statement = connection.createStatement();

        statement.executeUpdate( sql: "INSERT INTO videoGames (gameID, gameName, gameDesc, gameGenre, playerMode, price, PEGIrating, quantity)" +
            "VALUES(4, 'The Witcher 3: Wild Hunt', 'An open-world RPG that is goats', 'Open-world action-RPG', 'Single-player', 400, 18, 10)");

        statement.close();
        connection.close();
    } catch (Exception e) {
        System.out.println("Couldn't insert data to 'videoGames'");
    }
}
```

Figurbilde 27: Metode for å lage nye rader i tabellen "videoGames" gjennom bruken av “statement.executeUpdate”

Kalte på metoden i option 3 i main:

```
case 3:
    databaseConnection.createGame();
    System.out.println("What games do you want to order?: ");
    break;
```

Figurbilde 28: Metodekall av “createGame” fra “DatabaseConnection” klassen

Etter å kjørt option 3 og deretter option 1, så kommer dette frem i terminalen:

```
The Witcher 3: Wild Hunt
An open-world RPG that is goats
Open-world action-RPG
Single-player|
```

Figurbilde 29: Verdiene som ble skrevet inn i SQL-spørringen fra figurbilde 27, printet ut i terminalen

Bekreftet i mysql:

```
54
55 • select *
56 from videogames
57
```

gameID	gameName	gameDesc	gameGenre	playerMode	price
1	God Of War: Ragnarok	Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go	Action-adventure	Single-Player	899
2	Ghost Of Yotei	AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES	Action-adventure	Single-Player	699
3	Destiny 2	Dive into the world of Destiny 2 to explore the mysteries of the solar system and experience respon...	Frist-Person Looter-Shooter	Multiplayer	0
4	The Witcher 3: Wild Hunt	An open-world RPG that is goats	Open-world action-RPG	Single-player	400

Figurbilde 30: Bekreftelse på at “createGame” metoden klarte å lage en ny rad med de hardkoda verdiene i “videogames” tabellen

Har deretter laget en ny select metode kalt for “selectOrders”:

```

void selectOrders(){ 1 usage
    try{
        connection = DriverManager.getConnection(dbURL, username, password);

        Statement statement = connection.createStatement();

        ResultSet resultSet = statement.executeQuery( sql: "SELECT * FROM orders");

        while (resultSet.next()){
            System.out.println(resultSet.getInt( columnLabel: "orderID") + " \n" +
                               resultSet.getString( columnLabel: "customerFristName") + " \n" + resultSet.getString( columnLabel: "customerLastName") + "\n" +
                               resultSet.getString( columnLabel: "gameName"));
        }

        resultSet.close();
        statement.close();
        connection.close();
    }catch (Exception e){
        System.out.println("Failed to load data from table 'orders' ");
    }
}

```

Figurbilde 31: Koden for “selectOrders” metoden, der den lister opp alle ordre i “orders” tabellen.

Når brukeren trykker på case 2, så vil default ordenen bli skrevet ut i terminalen:

```

Welcome to online GameStop! What do you want to do?:
1. Show all available games
2. Show orders
3. Order a game
4. Return a game
5. Show customer info
6. Exit
2
Here are the current orders:
1344
John
Halo
God Of War: Ragnarok

```

Figurbilde 32: Utskriving av default ordenen når option 2 ble valgt i terminalen

I mysql (oppdatert versjon):

	orderID	customerFirstName	customerLastName	gameName	gameID	purchaseDate	returned
▶	1	Ahmed	Abdullah	Ghost Of Yotei	2	24-02-2026	0
	1344	John	Halo	God Of War: Ragnarok	1	28-02-2026	0

Figurbilde 33: Bekreftelse i mysql. Det er en ekstra ordre i bildet enn det som ble vist i figurbilde 32, og grunnen er fordi jeg ikke oppdaget at sql-skjermbildet etter å ha utført option 2 manglet og da var default verdien oppdatert for lenge siden.

Oppdaget også en bug i default casen, der den aktiveres hver gang brukerne trykker på tallet 6:

```
Welcome to online GameStop! What do you want to do?:
1. Show all available games
2. Show orders
3. Order a game
4. Return a game
5. Show customer info
6. Exit
6
Thank you for visiting online GameStop!
Invalid choice.
```

Figurbilde 34: Bug i default case, der den aktiveres hver gang brukeren slutter programmet. Fant ut at jeg glemte å legge til en “break” under “sout-delen” av option 6 (se figurbilde 9). Buggen ble fikset.

La også til en sjekk for å informere brukeren at du ikke kan skrive inn tekst i programmet. Dette forhindrer også at man får en “InputMismatchException” i terminalen:

```
if (scanner.hasNextInt()) {
    userChoice = scanner.nextInt();
    scanner.nextLine();
}else {
    String isValidInput = scanner.nextLine();
    System.out.println("Invalid input " + isValidInput + " Please try again.");
}
```

Figurbilde 35: If-else statement for å sjekke om brukeren har skrevet inn et tall eller tekst.

Dette blir resultatet når brukeren skriver inn tekst. Menyen kommer frem igjen og programmet gir brukeren en ny mulighet til å skrive inn riktig input. Dette er fordi programmet er i en while-loop og så lenge ikke option 6 er valgt, så vil programmet fortsette å prompte brukeren (se figurbilde 6):

```
adsfaefa
Invalid input adsfaefa Please try again.
Invalid choice.
Welcome to online GameStop! What do you want to do?:
1. Show all available games
2. Show orders
3. Order a game
4. Return a game
5. Show customer info
6. Exit
```

Figurbilde 36: Det invalide inputtet blir printet med en melding, hvor menyen kommer frem igjen for brukeren

Har nå også laget en “deleteGame” metode:

```
void deleteGame(){ 1 usage
    try{
        connection = DriverManager.getConnection(dbURL, username, password);
        Statement statement = connection.createStatement();

        statement.executeUpdate( sql: "DELETE from videoGames where gameID = 4");

        statement.close();
        connection.close();
    }catch (Exception e){
        System.out.println("Failed to delete data from table 'videoGames' ");
    }
}
```

Figurbilde 37: “deleteGame” metode, der den slette vekk spillet med ID = 4

Metoden ble plassert i option 4 i main:

```
case 4:
    System.out.println("What games do you want to return?: ");
    databaseConnection.deleteGame();
    break;
```

Figurbilde 38: Plassering av “deleteGame” metoden fra “DatabaseConnection” klassen

Før option 4 ble valgt (Witcher 3 har ID=4):

```
Here is all currently available games:
God Of War: Ragnarok
Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go
Action-adventure
Single-Player
Ghost Of Yotei
AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES
Action-adventure
Single-Player
Destiny 2
Dive into the world of Destiny 2 to explore the mysteries of the solar system and experience responsive first-person shooter combat. Unlock p
Frist-Person Looter-Shooter
Multiplayer
The Witcher 3: Wild Hunt
An open-world RPG that is goats
Open-world action-RPG
Single-player
```

Firgubilde 39: Utprinting av alle tilgjengelige spill, inkludert “The witcher 3”

I mysql:

```
55 • select *
56 from videogames
57
```

	gameID	gameName	gameDesc	gameGenre	playerMode	
▶	1	God Of War: Ragnarok	Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go	Action-adventure	Single-Player	8'
	2	Ghost Of Yotei	AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES	Action-adventure	Single-Player	6'
	3	Destiny 2	Dive into the world of Destiny 2 to explore the mysteries of the solar system and experience respon...	Frist-Person Looter-Shooter	Multiplayer	0
	4	The Witcher 3: Wild Hunt	An open-world RPG that is goats	Open-world action-RPG	Single-player	4

Firgubilde 40: Utlisting av alle spill i tabellen “videogames”

Dette er resultatet når brukerne trykker på option 4:

```
What games do you want to return?:
Welcome to online GameStop! What do you want to do?:
1. Show all available games
2. Show orders
3. Order a game
4. Return a game
5. Show customer info
6. Exit
```

Firgubilde 41: Fremvisning av teksten “What games do you want to return?” og meny etter at option 4 er valgt

Witcher 3 er slettet vekk fra listen:


```

Here is all currently available games:
God Of War: Ragnarok
Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go
Action-adventure
Single-Player
Ghost Of Yotei
AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES
Action-adventure
Single-Player
Destiny 2
Dive into the world of Destiny 2 to explore the mysteries of the solar system and experience responsive first-person shooter combat. Unlock power
First-Person Looter-Shooter
Multiplayer

```

Figurbilde 42: Resultatet etter å ha valgt option 4 og deretter option 1 for å liste opp spill. Witcher 3 er ikke lenger listet opp i terminalen

I mysql:

	gameID	gameName	gameDesc	gameGenre	playerMode	price	i
▶	1	God Of War: Ragnarok	Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go	Action-adventure	Single-Player	899	1
	2	Ghost Of Yotei	AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES	Action-adventure	Single-Player	699	1
	3	Destiny 2	Dive into the world of Destiny 2 to explore the mysteries of the solar system and experience respon...	First-Person Looter-Shooter	Multiplayer	0	1
*	NULL	NULL	NULL	NULL	NULL	NULL	0

Figurbilde 43: Bekreftelse på at “Witcher 3: Wild hunt” er også slettet vekk fra tabellen

Oppdaterte main filen slik at brukeren kan registrere og slette vekk customers:

```

while (userChoice != 8) {
    System.out.println("Welcome to online GameStop! What do you want to do?: ");
    System.out.println("1. Show all available games");
    System.out.println("2. Show orders");
    System.out.println("3. Order a game");
    System.out.println("4. Return a game");
    System.out.println("5 Register as a customer");
    System.out.println("6. Delete customer info");
    System.out.println("7. Show customer info");
    System.out.println("8. Exit");
}

```

Figurbilde 44: Utvidelse av menyen slik at brukeren kan registrere og slette vekk customers. Nå må brukeren trykke på 8 for å slutte programmet

(Jeg glemte å legge til metoden før koden ble oppdatert til å bruke “preparedStatement”. Denne metoden hadde samme oppsett som “createGame” metoden (henviser figurbilde 27), med egne kolonner og hardkoda verdier)

La til tilsvarende caser:

```
case 5:
    System.out.println("Registered as customer");
    databaseConnection.createCustomer();
    break;
case 6:
    System.out.println("Customer removed from database");
    databaseConnection.deleteCustomer();
    break;
```

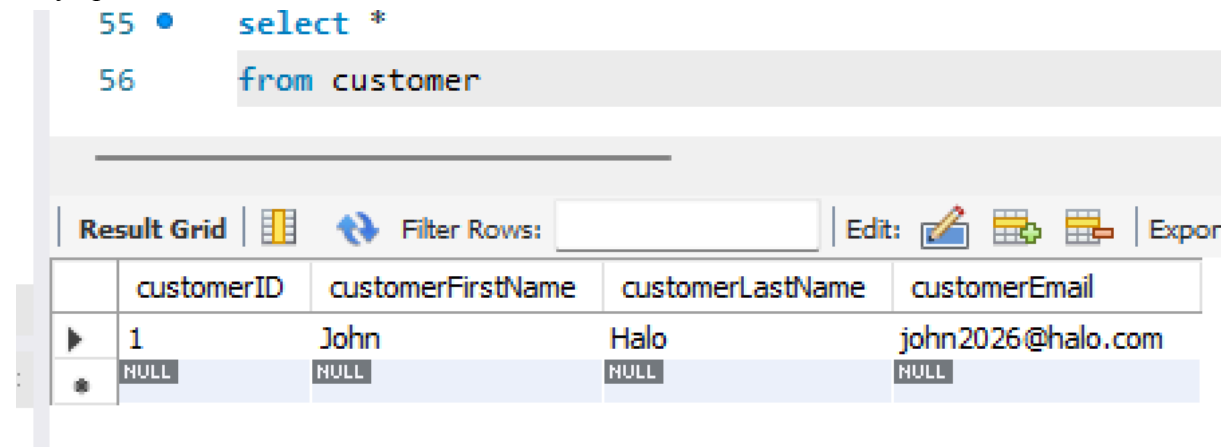
Figurbilde 45: Case 5 tillater brukeren å lage en ny bruker, ved at casen kaller på “createCustomer” metoden, mens case 6 sletter vekk en bruker gjennom "deleteCustomer" metoden.

Terminalen før option 5 er valgt:

```
Welcome to online GameStop! What do you want to do?:
1. Show all available games
2. Show orders
3. Order a game
4. Return a game
5 Register as a customer
6. Delete customer info
7. Show customer info
8. Exit
7
Here's the current customer info
John Halo
Welcome to online GameStop! What do you want to do?:
1. Show all available games
2. Show orders
3. Order a game
4. Return a game
5 Register as a customer
6. Delete customer info
7. Show customer info
8. Exit
```

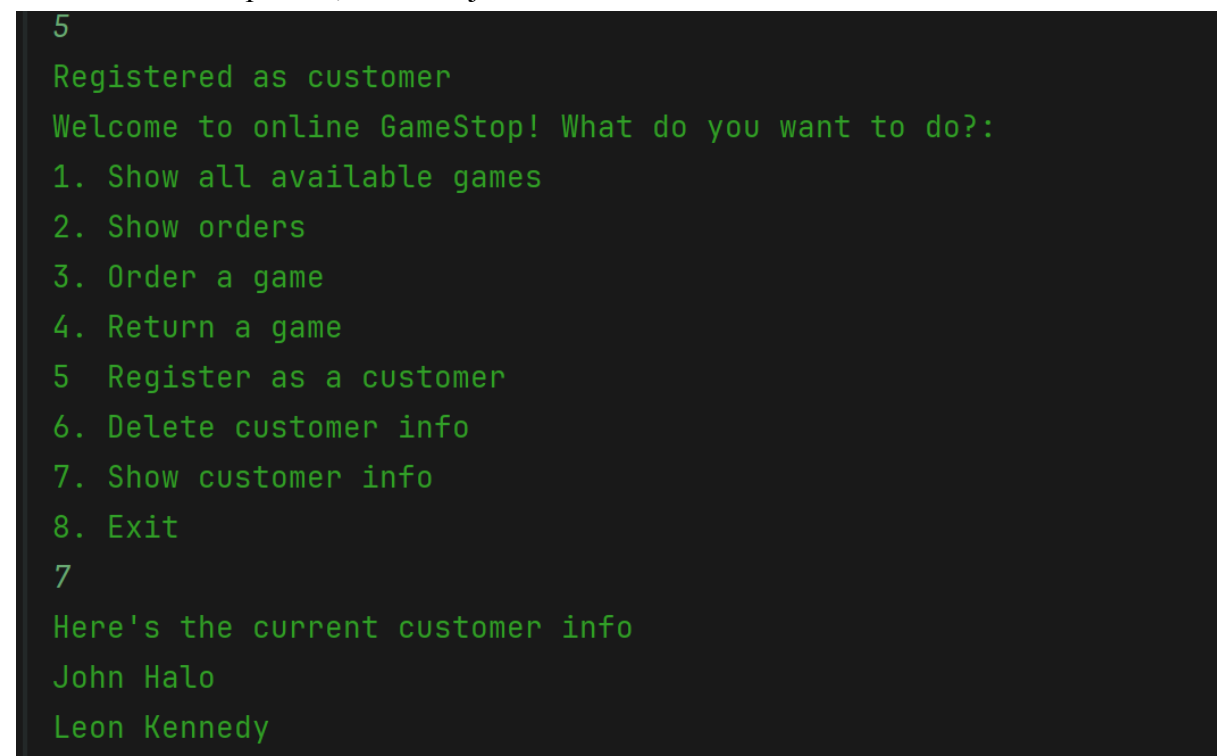
Figurbilde 46: Når man lister opp alle customers gjennom valg 7, så får man opp “John Halo” i terminalen, som er default verdien fra begynnelsen av prosjektet

I mysql:



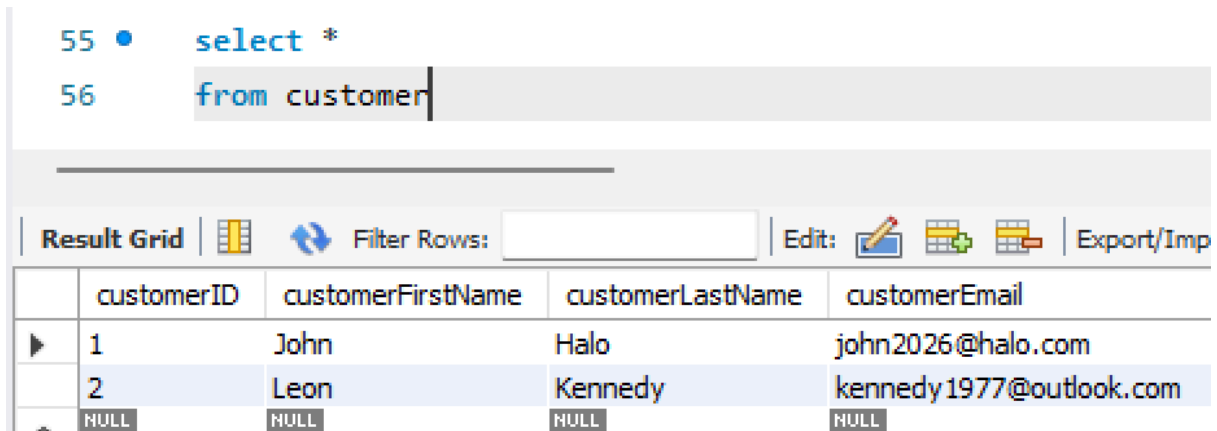
Figurbilde 47: Bare rad 1 med default-verdiene fra starten av prosjektet.

Terminalen etter option 5, når den kjører “createCustomer” metoden:



Figurbilde 48: Option 5 ble kjørt, programmet varslet “Registered as customer” og når option 7 ble kjørt, så fikk en listet opp både “John Halo” og “Leon Kennedy”

I databasen:



The screenshot shows a MySQL database interface. At the top, a SQL query is entered: `select * from customer`. Below the query, there is a toolbar with options like 'Result Grid', 'Filter Rows', 'Edit', and 'Export/Imp'. The results are displayed in a table with the following columns: `customerID`, `customerFirstName`, `customerLastName`, and `customerEmail`. The table contains two rows of data:

	customerID	customerFirstName	customerLastName	customerEmail
▶	1	John	Halo	john2026@halo.com
	2	Leon	Kennedy	kennedy1977@outlook.com
	NULL	NULL	NULL	NULL

Figurbilde 49: Bekreftelse i mysql at java klarte å legge til de harkoda verdiene fra “createCustomer” metoden i “customer” tabellen

Lagde en ny case kalt for “Create Order” i main:

```
while (userChoice != 9) {
    System.out.println("Welcome to online GameStop! What do you want to do?: ");
    System.out.println("1. Show all available games");
    System.out.println("2. Show orders");
    System.out.println("3. Create order");
    System.out.println("4. Order a game");
    System.out.println("5. Return a game");
    System.out.println("6. Register as a customer");
    System.out.println("7. Delete customer info");
    System.out.println("8. Show customer info");
    System.out.println("9. Exit");
}
```

Figurbilde 50: Plassert “Create Order” som case 3 og endret på rekkefølgen av casene, slik at exit er da case 9.

Dette er koden til “createOrders” metoden:

```
void createOrders(){
    try{
        connection = DriverManager.getConnection(dbURL, username, password);
        Statement statement = connection.createStatement();

        statement.executeUpdate( "INSERT into orders(orderID, customerFristName, customerLastName, gameName, gameID, purchaseData, returned)" +
            "VALUES(2, 'Jin', 'Sakai', 'Ghost Of Yotei', 2, '22-02-2026', false)");

        statement.close();
        connection.close();
    } catch (Exception e) {
        System.out.println("Couldn't insert data to 'orders'");
    }
}
```

Figurbilde 51: Koden for “createOrders”. Har også harkoda verdier og bruker statement, men dette ble fisket på senere

Før option 3:

```

9. Exit
2
Here are the current orders:
1344
John
Halo
God Of War: Ragnarok
Welcome to online GameStop! What do you want to do?:

```

Figurbilde 52: Når brukeren velger option 2, så lister terminalen opp default ordren fra starten av prosjektet.

I mysql:

	orderID	customerFristName	customerLastName	gameName	gameID	purchaseData	returned
▶	1344	John	Halo	God Of War: Ragnarok	1	28-02-2026	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figurbilde 53: Bekreftelse av default ordenen i mysql

Etter option 3:

```

3
Create a new order:
What games do you want to add?:
Welcome to online GameStop! What do you want to do?:
1. Show all available games
2. Show orders
3. Create order:
4. Order a game
5. Return a game
6. Register as a customer
7. Delete customer info
8. Show customer info
9. Exit
2
Here are the current orders:
2
Jin
Sakai
Ghost Of Yotei
1344
John
Halo
God Of War: Ragnarok

```

Figurbilde 54: Når brukeren trykker på option tre, så lages det en ny order. Når brukerne trykker på 2 igjen, så listes opp den nye ordenen. Det står “Create New order”, men det er fortsatt harkoda verdier.

I mysql:

	orderID	customerFristName	customerLastName	gameName	gameID	purchaseData	returned
▶	2	Jin	Sakai	Ghost Of Yotei	2	22-02-2026	0
	1344	John	Halo	God Of War: Ragnarok	1	28-02-2026	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figurbilde 55: Den nye ordren i “customer” tabellen. Den har en “orderID” = 2

Har også laget en “deleteOrders” metode, men jeg glemte å legge til et skjermbilde av metoden før den ble oppdatert til å bruke “preparedStatement”

Når option 4 ble valgt, så fikk man meldingen “Delete an order:”. Terminalen skriver også “What games do you want to add?” som kommer fra case 5, når man trykker på 5 i terminalen. Jeg hadde glemt å legge til en “break” statement i slutten av case 4

```
4
Delete an order
What games do you want to add?:
Welcome to online GameStop! What do you want to do?:
1. Show all available games
2. Show orders
3. Create order
4. Delete order
5. Order a game
6. Return a game
7. Register as a customer
8. Delete customer info
9. Show customer info
10. Exit
```

Figurbilde 56: Resultatet etter å ha trukket på option 4 i terminalen

Dette er resultatet i terminalen:

```

2
Here are the current orders:
1344
John
Halo
God Of War: Ragnarok

```

Figurbilde 58: orderID = 2 ble slettet vekk fra terminalen.

	orderID	customerFristName	customerLastName	gameName	gameID	purchaseData	returned
▶	1344	John	Halo	God Of War: Ragnarok	1	28-02-2026	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figurbilde 59: Bekreftelse i mysql

Lagde "WriteData" klassen og metoden "WriteIntoVideoGames":

```

import java.util.Scanner;

public class WriteData { no usages
    Scanner scanner = new Scanner(System.in); 2 usages
    DatabaseConnection databaseConnection = new DatabaseConnection();

    void writeIntoVideoGames(){ no usages
        System.out.println("Enter a game to wish for in the store: ");
        String gameName = scanner.nextLine();

        databaseConnection.createGame();
        scanner.close();
    }
}

```

Figurbilde 60: "WriteData" klasse med scanner og en "writeDataIntoVideoGames" metode som bruker scanneren

Endret innholdet i "createCustomer" klassen slik at den bruker "preparedStatement" i stedet for "statement". Er mer fleksibel og tillater brukeren å legge til ønsket data basert på placeholder verdiene. PreparedStatement ifølge dokumentasjon er også mer sikkert, da den setter opp en standard som SQL-querien må følge, noe som forhindrer at angripere kan injecte spesielle karakterer i koden:

```

void createCustomer(){ 1 usage
    try{
        connection = DriverManager.getConnection(dbURL, username, password);

        String createSQL = "INSERT INTO customer(customerID, customerFirstName, customerLastName, customerEmail) VALUES (?, ?, ?, ?)";

        PreparedStatement preparedStatement = connection.prepareStatement(createSQL);

        preparedStatement.setInt( parameterIndex: 1, x: 3);
        preparedStatement.setString( parameterIndex: 2, x: "John");
        preparedStatement.setString( parameterIndex: 3, x: "Doe");
        preparedStatement.setString( parameterIndex: 4, x: "johndoe1997@gmail.com");

        int rowsAffected = preparedStatement.executeUpdate();

        System.out.println(rowsAffected + " rows affected");

        preparedStatement.close();
        connection.close();
    }catch (Exception e){
        System.out.println("Failed to insert data into 'customer' ");
    }
}

```

Figurbilde 61: “createCustomer” metoden med bruk av “preparedstatement” i stedet for ”statement”

Oppdatert versjon av samme metode, der jeg har flytta ”preparedStatment” linjene til “writeDataIntoCustomer” metoden i “WriteData” klassen. Dette gjør det separasjon av ansvar mer tydelig da “createCustomer” metoden har som jobb da å bare lage nye rader, hvor dataen blir hentet fra “WriteData” klassen.:

```

void createCustomer(){ 1 usage
    try{
        connection = DriverManager.getConnection(dbURL, username, password);

        String createSQL = "INSERT INTO customer(customerID, customerFirstName, customerLastName, customerEmail) VALUES (?, ?, ?, ?)";

        PreparedStatement preparedStatement = connection.prepareStatement(createSQL);

        writeData.writeDataIntoCustomer(preparedStatement);

        int rowsAffected = preparedStatement.executeUpdate();

        System.out.println(rowsAffected + " rows affected");

        preparedStatement.close();
        connection.close();
    }catch (Exception e){
        System.out.println("Failed to insert data into 'customer' ");
    }
}

```

Figurbilde 62: Flyttet vekk “preparedStatement”-blokken, lagde en ny instanse av “WriteData” klassen, der en kaller på metoden "writeDataIntoCustomer". Henviser figurbilde 63


```
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;

public class WriteData { 2 usages
    Scanner scanner = new Scanner(System.in); 4 usages

    void writeDataIntoCustomer(PreparedStatement preparedStatement) throws SQLException {
        System.out.print("Enter customer info: (ID, firstName, lastName, email) ");

        int customerID = scanner.nextInt();
        String customerFirstName = scanner.nextLine();
        String customerLastName = scanner.nextLine();
        String customerEmail = scanner.nextLine();

        preparedStatement.setInt( parameterIndex: 1, customerID);
        preparedStatement.setString( parameterIndex: 2, customerFirstName);
        preparedStatement.setString( parameterIndex: 3, customerLastName);
        preparedStatement.setString( parameterIndex: 4, customerEmail);

        preparedStatement.executeUpdate();
    }
}
```

Figurbilde 63: Oppdatert kode i “WriteData” klassen

Når jeg velger option 7 igjen, så kan brukeren endelig skrive inn data selv:

```
7
Registered as customer
Enter customer info: (ID, firstName, lastName, email) 5
Ahmed
Abdulla
```

Figurbilde 64: Input av verdier etter at valg 7 er oppgitt.

Og når man trykker på option 9, så kommer dataen fram i terminalane og i sql:

```
9
Here's the current customer info:
John Halo john2026@halo.com
John Doe johndoe1997@gmail.com
Jambo Bwana jamboBwana1998@gmail.com
Ahmed Abdulla
```

Figurbilde 65: Oppdatert “customer info” i terminalen

I MySQL:

	customerID	customerFirstName	customerLastName	customerEmail
▶	1	John	Halo	john2026@halo.com
	3	John	Doe	johndoe1997@gmail.com
	4	Jambo	Bwana	jamboBwana1998@gmail.com
	5	Ahmed	Abdulla	Abdulla
*	NULL	NULL	NULL	NULL

Figurbilde 66: Bekreftet innsetting av data til “customer” tabellen

Det er en tydelig feil i programmet, hvor denskipper over en linje etter “customerID” og verdiene skrevet over blir skjøvet til høyre. Dette er mest sannsynlig et newline problem.

Fikset feilen ved å legge til en “scanner.nextLine()” etter “int customerID = scanner.nextInt()”:

```
int customerID = scanner.nextInt();
scanner.nextLine();
String customerFirstName = scanner.nextLine();
String customerLastName = scanner.nextLine();
String customerEmail = scanner.nextLine();
```

Figurbilde 67: La til “scanner.nextLine()” under “int customerID”

Når jeg gjorde samme test igjen, så ble dataene skrevet i terminal plassert riktig i databasen:

	customerID	customerFirstName	customerLastName	customerEmail
▶	1	John	Halo	john2026@halo.com
	3	John	Doe	johndoe1997@gmail.com
	4	Jambo	Bwana	jamboBwana1998@gmail.com
	5	Ahmed	Abdullah	ahmed030503@gmail.com
	NULL	NULL	NULL	NULL

Figurbilde 68: Forskyvningen til høyre når customer info blir lagt til er fjernet vekk
Har laget metode "WriteDataIntoVideoGames":

```

void writeDataIntoVideoGames(PreparedStatement preparedStatement) throws SQLException { 1 usage
    System.out.println("Enter game info: (gameID, gameName, gameDesc, gameGenre, playerMode, price, PEGIrating, quantity)");

    int gameID = scanner.nextInt();
    scanner.nextLine();
    String gameName = scanner.nextLine();
    String gameDesc = scanner.nextLine();
    String gameGenre = scanner.nextLine();
    String playerMode = scanner.nextLine();
    int price = scanner.nextInt();
    int PEGIrating = scanner.nextInt();
    int quantity = scanner.nextInt();

    preparedStatement.setInt( parameterIndex: 1, gameID);
    preparedStatement.setString( parameterIndex: 2, gameName);
    preparedStatement.setString( parameterIndex: 3, gameDesc);
    preparedStatement.setString( parameterIndex: 4, gameGenre);
    preparedStatement.setString( parameterIndex: 5, playerMode);
    preparedStatement.setInt( parameterIndex: 6, price);
    preparedStatement.setInt( parameterIndex: 7, PEGIrating);
    preparedStatement.setInt( parameterIndex: 8, quantity);

    preparedStatement.executeUpdate();

    scanner.close();
}

```

Figurbilde 69: Innholdet til metoden “writeDataIntoVideoGames”

Metoden over gjør det mulig for brukeren å legge til spill gjennom terminalen og inn i databasen:

```

5
What games do you want to add?:
Enter game info: (gameID, gameName, gameDesc, gameGenre, playerMode, price, PEGIrating, quantity)
5
Bloodborne
A lone traveller. A cursed town. A deadly mystery that swallows everything it touches
Soulslike
Single-player
219
16
15

```

Figurbilde 70: Input av verdier etter at option 5 er valgt. Verdiene er basert på det som står i parentes

Dette er bekreftet og plassert riktig i mysql:

5	Bloodborne	A lone traveller. A cursed town. A deadly mystery that swallows everything it touches	Soulslike	Single-player	219
---	------------	---	-----------	---------------	-----

Figurbilde 71: Verdiene fra figurbilde 70 ble korrekt implementert i databasen

Det fungerer, men for en eller annen årsak så stopper programmet og gir “No line break” etter at man har inputtet verdiene til terminalen.

Problemet var at jeg skrev "quanity" i stedet for “quantity”, så Java klarte ikke skjønne hva det betydde noe som gjorde at jeg ikke kunne skrive inn resultatet og da fikk det ovennevnte feilen:

```

+ resultSet.getInt( columnLabel: "quanity") );

```

Figurbilde 72: Årsaken til at java kasta “No line break” feilen

Oppdaterte også koden i “selectGame” metoden slik at den skal vise alle kolumnene i tabellen “videoGames”, ved å loope gjennom kolumnene:

```
while (resultSet.next()){
    System.out.println(resultSet.getInt( columnLabel: "gameID") + "\n" +
        resultSet.getString( columnLabel: "gameName") + " \n" + resultSet.getString( columnLabel: "gameDesc")
        + " \n" + resultSet.getString( columnLabel: "gameGenre") + " \n" + resultSet.getString( columnLabel: "playerMode") + " \n" +
        resultSet.getInt( columnLabel: "price") + " \n" + resultSet.getInt( columnLabel: "PEGirating") + " \n" + resultSet.getInt( columnLabel: "quantity" ));
}
```

Figurbilde 73: Bruk av “resultSet” for å printe alle kolonnen fra “videogames” tabellen.

Har også nå laget en “deleteDataFromCustomer” metode. Denne metoden sletter vekker kunder fra tabellen “customer”, der brukerne trenger bare å skrive “customerID”:

```
void deleteDataFromCustomer(PreparedStatement preparedStatement) throws SQLException {
    System.out.println("Which customer do you want to remove?: (customerID)" );

    int customerID = scanner.nextInt();

    preparedStatement.setInt( parameterIndex: 1, customerID);

    preparedStatement.executeUpdate();
}
```

Figurbilde 74: Metode for “deleteDataFromCustomer”

Alle customers i “customer” tabellen før option 8:

```
7
Registered as customer
Enter customer info: (ID, firstName, lastName, email)
6
Geralt
Rivia
kaerMorhen1240@yahoo.com
```

Figurbilde 75: Lagde en ny customer “Geralt Rivia” med customerID = 6 og mailen “kaerMorhen1240@yahoo.com”

Fikk det nye customer info printet ut i terminal og i mysql:

```
Here's the current customer info:  
1 John Halo john2026@halo.com  
2 Sarosh Bekir sarosh95@hotmail.com  
3 John Doe johndoe1997@gmail.com  
4 Jin Sakai jinSakai1277@gmail.com  
5 Ahmed Abdullah ahmed030503@gmail.com  
6 Geralt Rivia kaerMorhen1240@yahoo.com
```

Figurbilde 76: Inputen fra figurbilde 75 ble registrert inn i terminalen

	customerID	customerFirstName	customerLastName	customerEmail
►	1	John	Halo	john2026@halo.com
	2	Sarosh	Bekir	sarosh95@hotmail.com
	3	John	Doe	johndoe1997@gmail.com
	4	Jin	Sakai	jinSakai1277@gmail.com
	5	Ahmed	Abdullah	ahmed030503@gmail.com
	6	Geralt	Rivia	kaerMorhen1240@yahoo.com

Figurbilde 77: Bekrefter korrekt input i mysql

Valgte option 8 og skrev inn 6 som svar. Geralt har “customerID = 6” som vist i figurbilde 77. Dere kan ignorere “No customer with inputted ID found” meldingen, det er en bug som ikke har blitt fisket ennå. Geralt har blitt fjernet fra “customer” tabellen som vist i figurbilde 79 og 80

```
8  
Customer removed from database  
Which customer do you want to remove?: (customerID)  
6  
No customer with inputted ID found
```

Figurbilde 78: Fjernet vekk customer med ID 6

```
9
Here's the current customer info:
1 John Halo john2026@halo.com
2 Sarosh Bekir sarosh95@hotmail.com
3 John Doe johndoe1997@gmail.com
4 Jin Sakai jinSakai1277@gmail.com
5 Ahmed Abdullah ahmed030503@gmail.com
```

Figurbilde 79: Alle customers i tabellen “customer” når option 9 er valgt

I mysql:

	customerID	customerFirstName	customerLastName	customerEmail
▶	1	John	Halo	john2026@halo.com
	2	Sarosh	Bekir	sarosh95@hotmail.com
	3	John	Doe	johndoe1997@gmail.com
	4	Jin	Sakai	jinSakai1277@gmail.com
	5	Ahmed	Abdullah	ahmed030503@gmail.com

Figurbilde 80: Bekreftet i mysql

Har også laget en metode kalt for “deleteDataFromVideoGames”:

```
void deleteDataFromVideoGames(PreparedStatement preparedStatement) throws SQLException {
    System.out.println("Which game do you want to remove?: (gameID)" );

    int gameID = scanner.nextInt();

    preparedStatement.setInt( parameterIndex: 1, gameID);

    preparedStatement.executeUpdate();
}
```

Figurbilde 81: Metode for å slette vekk dataspill fra tabellen “videogames”, gjennom bruken av kolonnen “gameID”

Når man velger option 6, så kan en skrive in ID til spillet (i denne situasjonen, er ID = 5 spillet Bloodborne):

Før option 6:

	gameID	gameName	gameDesc	gameGenre
▶	1	God Of War: Ragnarok	Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go	Action-adver
	2	Ghost Of Yotei	AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES	Action-adver
	3	Destiny 2	Dive into the world of Destiny 2 to explore the mysteries of the solar system and experience respon...	Frist-Person
	4	The Witcher 3: Wild Hunt	An open-world RPG that is goated	Open-world
	5	Bloodborne	A lone traveller. A cursed town. A deadly mystery that swallows everything it touches	Soulslike

Figurbilde 82: Bloodorne med gameID = 5 er i tabellen “videoGames”

Når man velger option 6, så blir man promptet om å slette vekk et spill ved å skrive inn spillets ID. Samme greie som med figurbilde 78, teksten “No game with inputted game ID found” er en bug og bloodborne ble slettet vekk fra databasen som vist i figurbilde 84:

```

6
What games do you want to return?:
Which game do you want to remove?: (gameID)
5
No game with inputted game ID found

```

Figurilde 83: Prompten brukeren får når man velger å slette et spill, der tallet 5 er skrevet.

Etter option 6:

	gameID	gameName	gameDesc	gameGenre
▶	1	God Of War: Ragnarok	Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go	Action-adventure
	2	Ghost Of Yotei	AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES	Action-adventure
	3	Destiny 2	Dive into the world of Destiny 2 to explore the mysteries of the solar system and experience respon...	Frist-Person Loot
	4	The Witcher 3: Wild Hunt	An open-world RPG that is goated	Open-world action

Figurbilde 84: GameID 5 ble slettet vekk fra tabellen “videogames”

Fikset på kolonnene “purchaseData” til “purchaseDate” og “customerFristName” til “customerFirstName” i tabellen “orders” i mysql. Samme fiks ble også gjort i java:

```

/*
ALTER TABLE orders
RENAME COLUMN purchaseData to purchaseDate*/

```

orderID	customerFristName	customerLastName	gameName	gameID	purchaseDate	returned
1344	John	Halo	God Of War: Ragnarok	1	28-02-2026	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```

58  /*
59  ALTER TABLE orders
60  RENAME COLUMN customerFristName to customerFirstName*/
61

```

	orderID	customerFirstName	customerLastName	gameName	gameID	purchaseDate	returned
▶	1344	John	Halo	God Of War: Ragnarok	1	28-02-2026	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figurbilde 85 og 86: Fikset typo feil i “customer” tabellen

Lagde en ny metode kalt for “writeDataIntoOrders”:

```

void writeDataIntoOrders(PreparedStatement preparedStatement) throws SQLException { 1 usage
    System.out.println("Enter order info: (orderID, customerFirstName, customerLastName, gameName, gameID, purchaseDate, returned)");

    int orderID = scanner.nextInt();
    scanner.nextLine();
    String customerFirstName = scanner.nextLine();
    String customerLastName = scanner.nextLine();
    String gameName = scanner.nextLine();
    int gameID = scanner.nextInt();
    String purchaseDate = scanner.nextLine();
    scanner.nextLine();
    boolean returned = scanner.nextBoolean();

    preparedStatement.setInt( parameterIndex: 1, orderID);
    preparedStatement.setString( parameterIndex: 2, customerFirstName);
    preparedStatement.setString( parameterIndex: 3, customerLastName);
    preparedStatement.setString( parameterIndex: 4, gameName);
    preparedStatement.setInt( parameterIndex: 5, gameID);
    preparedStatement.setString( parameterIndex: 6, purchaseDate);
    preparedStatement.setBoolean( parameterIndex: 7, returned);
}

```

Figurbilde 87: Metode for å skrive inn orders til databasen

Modifiserte “createOrders” metoden, slik at den bruker “preparedStatement” i stedet for “statement”:

```

void createOrders(){ 1 usage
    try{
        connection = DriverManager.getConnection(dbURL, username, password);

        String createSQL = "INSERT into orders(orderID, customerFirstName, customerLastName, gameName, gameID, purchaseDate, returned) "
            + "values (?, ?, ?, ?, ?, ?, ?)";

        PreparedStatement preparedStatement = connection.prepareStatement(createSQL);

        writeData.writeDataIntoOrders(preparedStatement);
        int rowsAffected = preparedStatement.executeUpdate();

        System.out.println(rowsAffected + " rows affected");

        preparedStatement.close();
        connection.close();
    } catch (Exception e) {
        System.out.println("Couldn't insert data to 'orders'");
    }
}

```

Figurbilde 88: Fjernet vekk hardkoda verdier i koden og gjorde slik at “createOrders” kalte på metoden “writeDataIntoOrders” fra “WriteData” for motta input

Valgte option 3, der jeg kunne skrive inn en order:

```
3
Create an new order:
Enter order info: (orderId, customerFirstName, customerLastName, gameName, gameId, purchaseDate, returned)
1
Ahmed
Abdullah
Ghost Of Yotei
2
23-03-2026
false
1 rows affected
```

Figurbilde 89: Input av en order gjennom terminalen etter å ha valgt option 3

Når option 2 ble valgt, så får jeg dette opp::

```
2
Here are the current orders:
1
Ahmed
Abdullah
Ghost Of Yotei
2

false
1344
John
Halo
God Of War: Ragnarok
1
28-02-2026
false
```

Figurbilde 90: Liste av alle ordere, inkludert den nye ordere inputtet fra figurbilde 89

I mysql:

	orderID	customerFirstName	customerLastName	gameName	gameID	purchaseDate	returned
►	1	Ahmed	Abdullah	Ghost Of Yotei	2		0

Figurbilde 91: Bekreftet i mysql

Fikk problem med koden der hverken terminalen eller mysql viste frem “purchaseDate”. Etter en gjennomgang av “writeDateIntoOrders” metoden så var det slik at jeg feilplasserte “scanner.nextLine()” i rekkefølgen:

```

void writeDataIntoOrders(PreparedStatement preparedStatement) throws SQLException {
    System.out.println("Enter order info: (orderId, customerFirstName, customerLastName, gameName, gameId, purchaseDate, returned)");

    System.out.print("Order ID: ");
    int orderId = scanner.nextInt();
    scanner.nextLine();
    System.out.print("First Name: ");
    String customerFirstName = scanner.nextLine();
    System.out.print("Last Name: ");
    String customerLastName = scanner.nextLine();
    System.out.print("Game Name: ");
    String gameName = scanner.nextLine();
    System.out.print("Game ID: ");
    int gameId = scanner.nextInt();
    System.out.print("Purchase Date: ");
    scanner.nextLine();
    String purchaseDate = scanner.nextLine();
    //scanner.nextLine();
    System.out.print("Returned: ");
    boolean returned = scanner.nextBoolean();
}

```

Figurbilde 92: Utkommentere “scanner.nextLine()” under “String purchaseDate” og plasserte en ny en over “String purchaseDate”

I mysql:

	orderId	customerFirstName	customerLastName	gameName	gameID	purchaseDate	returned
▶	1	Ahmed	Abdullah	Ghost Of Yotei	1	25-11-2025	0
	2	Sarosh	Bekir	The Witcher 3: Wild Hunt	4	25-03-2026	1

Figurbilde 93: Nå ble inputted “purchaseDate” verdiene også plassert riktig i tabellen

Har også laget en “deleteDataFromOrders” metode i koden:

```

void deleteDataFromOrders(PreparedStatement preparedStatement) throws SQLException {
    System.out.println("Which order do you want to remove?: (orderId)" );

    int orderId = scanner.nextInt();

    preparedStatement.setInt( parameterIndex: 1, orderId);

    preparedStatement.executeUpdate();
}

```

Figurbilde 94: Metode for å slette vekk ordre, der man baserer seg ut ifra “orderId”

Modifiserte "deleteOrders" metoden slik som de andre delete metodene fra før:

```
void deleteOrders(){ 1 usage
    try {
        connection = DriverManager.getConnection(dbURL, username, password);

        String deleteSQL = "DELETE FROM orders WHERE orderID = ?";

        PreparedStatement preparedStatement = connection.prepareStatement(deleteSQL);

        writeData.deleteDataFromOrders(preparedStatement);

        int deletedRowsAffected = preparedStatement.executeUpdate();

        if (deletedRowsAffected > 0 ){
            System.out.println(deletedRowsAffected + " rows affected");
        }else {
            System.out.println("No orders with inputted order ID found");
        }

        preparedStatement.close();
        connection.close();
    }catch (Exception e){
        System.out.println("Couldn't delete order from table 'orders' ");
    }
}
```

Figurbilde 95: “deleteOrders” metode med samme oppsett som de andre “delete”-metodene.

Før option 4 ble valgt:

```
2
Here are the current orders:
1
Ahmed
Abdullah
Ghost Of Yotei
2

false
1344
John
Halo
God Of War: Ragnarok
1
28-02-2026
false
```

Figurbilde 96: Både ordreID = 1 og ordreID = 1344 kan vises i terminalen

I mysql:

	orderID	customerFirstName	customerLastName	gameName	gameID	purchaseDate	returned
▶	1	Ahmed	Abdullah	Ghost Of Yotei	2	24-02-2026	0
	1344	John	Halo	God Of War: Ragnarok	1	28-02-2026	0

Figurbilde 97: Det samme gjelder i mysql

Etter option 4. Ignorer feilmeldingen i bunnen, ordre ble slettet som vist i figurbilde 98 og 99:

```

4
Delete an order
Which order do you want to remove?: (orderID)
1
No orders with inputted order ID found

```

Figurbilde 97: Sletter vekk ordre med orderID = 1.

```

2
Here are the current orders:
1344
John
Halo
God Of War: Ragnarok
1
28-02-2026
false

```

Figurbilde 98: Bare ordreID = 1344 er tilgjengelig i terminalen og i mysql.

I mysql:

	orderID	customerFirstName	customerLastName	gameName	gameID	purchaseDate	returned
▶	1344	John	Halo	God Of War: Ragnarok	1	28-02-2026	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figurbilde 99: OrderID = 1 ble også slettet i tabellen “orders” i mysql

Har også laget en “updateDataFromCustomer”:

```
void updateDataFromCustomer(PreparedStatement preparedStatement) throws SQLException { 1 usage
    System.out.println("Who's customer info do you want to update?: (customerFirstName, customerLastName, Email, customerID)");

    String customerFirstName = scanner.nextLine();
    String customerLastName = scanner.nextLine();
    String customerEmail = scanner.nextLine();
    int customerID = scanner.nextInt();

    preparedStatement.setString( parameterIndex: 1, customerFirstName);
    preparedStatement.setString( parameterIndex: 2, customerLastName);
    preparedStatement.setString( parameterIndex: 3, customerEmail);
    preparedStatement.setInt( parameterIndex: 4, customerID);
}
```

Figurbilde 100: Metode for å oppdatere eksisterende kundeinfo, basert på bruker input

Lagde deretter en “updateCustomer” metod for å bruke “updateDataFromCustomer” metoden:

```
void updateCustomer(){ 1 usage
    try{
        connection = DriverManager.getConnection(dbURL, username, password);

        String updateSQL = "UPDATE customer SET customerFirstName = ?, customerLastName = ?, customerEmail = ? WHERE customerID = ?";

        PreparedStatement preparedStatement = connection.prepareStatement(updateSQL);

        writeData.updateDataFromCustomer(preparedStatement);

        int updatedRows = preparedStatement.executeUpdate();

        if (updatedRows > 0) {
            System.out.println(updatedRows + " rows affected");
        } else {
            System.out.println("No customer with matching inputted ID ");
        }

        preparedStatement.close();
        connection.close();
    } catch (Exception e){
        System.out.println("Failed to update data into 'customer' ");
        e.printStackTrace();
    }
}
```

Figurbilde 101: Metoden som bruker “updateDataFromCustomer”, der den sender oppdatert info til databasen etter brukerinput

Lagde en ny case, slik at jeg kan aktivere kodeblokken over:

```
case 8:
    System.out.println("Customer info updated: ");
    databaseConnection.updateCustomer();
    break;
```

Figurbilde 102: Plassering av “updateCustomer” metoden fra “DatabaseConnection” klassen

Dette er resultatet:

```
8
Customer info updated:
Who's customer info do you want to update?: (customerFirstName, customerLastName, Email, customerID)
Geralt
Rivia
kaerhmorhen@gmail.com
4
1 rows affected
```

Figurbilde 103: Inputtet verdiene som trengtes for å oppdatere kundeinformasjon og fikk det bekreftet med “1 rows affected”

I terminalen og i mysql: (ID = 4 hadde “Jin Sakai” og email = jinsakai1277@gmail.com)

```
10
Here's the current customer info:
1 John Halo johnhalo@gmail.com
2 Atsu Onryo onryo1640@hotmail.com
3 John Doe johndoe1997@gmail.com
4 Geralt Rivia kaerhmorhen@gmail.com
5 Ahmed Abdullah ahmed030503@gmail.com
```

Figurbilde 104: Kundeinfo, hvor ID = 4 var endret

	4	Geralt	Rivia	kaerhmorhen@gmail.com
----------------------	---	--------	-------	-----------------------

Figurbilde 105: Bekreftet det i mysql

Lagde en update metode for videogames tabellen kalt for “updateDataFromVideoGames”:

```
void updateDataFromVideoGames(PreparedStatement preparedStatement) throws SQLException { 1 usage
    System.out.println("Which videogame info do you want to update? (gameName, gameDesc, gameGenre, playerMode, price, PEGIrating, quantity, gameID");

    String gameName = scanner.nextLine();
    String gameDesc = scanner.nextLine();
    String gameGenre = scanner.nextLine();
    String playerMode = scanner.nextLine();
    int price = scanner.nextInt();
    int PEGIrating = scanner.nextInt();
    int quantity = scanner.nextInt();
    int gameID = scanner.nextInt();

    preparedStatement.setString( parameterIndex: 1, gameName);
    preparedStatement.setString( parameterIndex: 2, gameDesc);
    preparedStatement.setString( parameterIndex: 3, gameGenre);
    preparedStatement.setString( parameterIndex: 4, playerMode);
    preparedStatement.setInt( parameterIndex: 5, price);
    preparedStatement.setInt( parameterIndex: 6, PEGIrating);
    preparedStatement.setInt( parameterIndex: 7, quantity);
    preparedStatement.setInt( parameterIndex: 8, gameID);
}
```

Figurbilde 106: Metode for å oppdatere eksisterende spillinfo, basert på brukerinput

Lagde deretter en metode kalt for “updateGame” for å bruke “updateDataFromVideoGames”:

```
void updateGame(){ 1 usage
    try{
        connection = DriverManager.getConnection(dbURL, username, password);

        String updateSQL = "UPDATE videogames set gameName = ?, gameDesc = ?, gameGenre = ?, playerMode = ?, price = ?, PEGIrating = ?, quantity = ? WHERE gameID = ? ";

        PreparedStatement preparedStatement = connection.prepareStatement(updateSQL);

        writeData.updateDataFromVideoGames(preparedStatement);

        int updatedRows = preparedStatement.executeUpdate();

        if (updatedRows > 0) {
            System.out.println(updatedRows + " rows affected");
        }else {
            System.out.println("No game with matching inputted ID ");
        }

        preparedStatement.close();
        connection.close();
    }catch (Exception e){
        System.out.println("Failed to update data from table 'videogames' ");
    }
}
```

Figurbilde 107: Metoden som bruker “updateDataFromVideoGames”, der den sender oppdatert info til databasen etter brukerinput

Lagde en ny case for å inkludere “updateGame” metoden i main menu:


```

        break;
    case 6:
        System.out.println("What games do you want to modify?: ");
        databaseConnection.updateGame();
        break;

```

Figurbilde 108: Plasserte “updateGame” fra “DatabaseConnection” i case 6

Dette er resultatet i terminalen:

```

6
What games do you want to modify?:
Which videogame info do you want to update? (gameName, gameDesc, gameGenre, playerMode, price, PEGIrating, quantity, gameID)
Genshin Impact
Anime waifu collector
Open-world
Single-player
0
12
40
3

```

Figurbilde 109: Inputtet nye verdier for gameID = 3

Bekreftelse på endring i mysql:

	gameID	gameName	gameDesc	gameGenre	playerMode	pr
▶	1	God Of War: Ragnarok	Embark on an epic and heartfelt journey as Kratos and Atreus struggle with holding on and letting go	Action-adventure	Single-Player	89
	2	Ghost Of Yotei	AT THE NORTHERN EDGE OF JAPAN, A NEW WARRIOR RISES	Action-adventure	Single-Player	69
	3	Genshin Impact	Anime waifu collector	Open-world	Single-player	0
	4	The Witcher 3: Wild Hunt	An open-world RPG that is goats	Open-world action-RPG	Single-player	40
	5	Ghost Of Tsushima	Something	Action-adventure	Single-player	89

Figurbilde 110: gameID = 3 er også oppdatert med nye verdier

Lagde en ny metode kalt for “updateDataFromOrders”:

```

void updateDataFromOrders(PreparedStatement preparedStatement) throws SQLException {
    System.out.println("Which order info do you want to update? (customerFirstName, customerLastName, gameName, purchaseDate, returned, gameID, orderID)");

    String customerFirstName = scanner.nextLine();
    String customerLastName = scanner.nextLine();
    String gameName = scanner.nextLine();
    String purchaseDate = scanner.nextLine();
    boolean returned = scanner.nextBoolean();
    int gameID = scanner.nextInt();
    int orderID = scanner.nextInt();

    preparedStatement.setString(1, customerFirstName);
    preparedStatement.setString(2, customerLastName);
    preparedStatement.setString(3, gameName);
    preparedStatement.setString(4, purchaseDate);
    preparedStatement.setBoolean(5, returned);
    preparedStatement.setInt(6, gameID);
    preparedStatement.setInt(7, orderID);
    preparedStatement.executeUpdate();
}

```

Figurbilde 111: Metode som oppdaterer eksisterende info i “orders” tabellen.

Lagde deretter en tilsvarende metode kalt for “updateOrders” hvor jeg kalte metoden “updateDataFromOrders” til:

```
void updateOrders(){ 1 usage
    try{
        connection = DriverManager.getConnection(dbURL, username, password);

        String updateSQL = "UPDATE orders set customerFirstName = ?, customerLastName = ?, gameName = ?, purchaseDate = ?, returned = ?, gameID = ? WHERE orderID = ?";

        PreparedStatement preparedStatement = connection.prepareStatement(updateSQL);

        writeData.updateDataFromOrders(preparedStatement);

        int updatedRows = preparedStatement.executeUpdate();

        if (updatedRows > 0) {
            System.out.println(updatedRows + " rows affected");
        }else {
            System.out.println("No orders with matching inputted ID");
        }
    }catch (Exception e){
        System.out.println("Failed to update data from table 'orders' ");
        e.printStackTrace();
    }
}
```

Figurbilde 112: Metoden som bruker “updateDateFromOrders”, der den sender oppdatert info til databasen etter brukerininput

Modifiserte casenummerne slik at jeg kan bruke metoden over i case 4:

```
case 4:
    System.out.println("Update an order: ");
    databaseConnection.updateOrders();
    break;
```

Figurbilde 113: Plasserte “updateOrders” fra “DatabaseConnection” i case 4

I terminalen når en velger option 4:

```
4
Update an order:
Which order info do you want to update? (customerFirstName, customerLastName, gameName, purchaseDate, returned, gameID, orderID)
Kassim
Mohammed
God of War: Ragnarok
24-03-2026
false
1
2
1 rows affected
```

Figurbilde 114: Input av verdier etter å valgt option 4

I mysql:

	orderID	customerFirstName	customerLastName	gameName	gameID	purchaseDate	returned
►	1	Ahmed	Abdullah	Ghost Of Yotei	2		0
	2	Kassim	Mohammed	God of War: Ragnarok	1	24-03-2026	0

Figurbilde 115: Lagret inputted verdier fra figurbilde 114 i mysql

Jeg har også nå lagt til ekstra printlines inn de forskjellige operasjonene slik at det er lettere for brukeren å vite hva de skal skrive til hver kolonne. Dette er gjort for alle CREATE, UPDATE og DELETE metoder i programmet. Ta dette som eksempel:

```
void writeDataIntoCustomer(PreparedStatement preparedStatement) throws SQLException {  
    System.out.println("Enter customer info: (ID, firstName, lastName, email) ");  
  
    System.out.println("Customer ID: " );  
    int customerID = scanner.nextInt();  
    scanner.nextLine();  
    System.out.println("Enter customer first name: ");  
    String customerFirstName = scanner.nextLine();  
    System.out.println("Enter customer last name: ");  
    String customerLastName = scanner.nextLine();  
    System.out.println("Enter customer email: ");  
    String customerEmail = scanner.nextLine();  
}
```

Figurbilde 116: Plassert printlines før scanner input slik at brukerne vet hva de skal skrive inn i hver kolonne.

For å gjøre det lettere for brukeren å vite hvilke ordre de ønsker å modifisere eller legge til gjennom UPDATE, DELETE og CREATE metodene, så har jeg kalt på SELECT metodene for hver tabell og plassert dem i korresponderende cases:

```
case 3:
    System.out.println("Here are the current orders: ");
    databaseConnection.selectOrders();

    System.out.println(" ");

    System.out.println("Create an new order: ");
    databaseConnection.createOrders();
    break;
case 4:
    System.out.println("Here are the current orders: ");
    databaseConnection.selectOrders();

    System.out.println(" ");

    System.out.println("Update an order: ");
    databaseConnection.updateOrders();
    break;
case 5:
    System.out.println("Here are the current orders: ");
    databaseConnection.selectOrders();

    System.out.println(" ");

    System.out.println("Delete an order");
    databaseConnection.deleteOrders();
    break;
```

Figurbilde 117: Kalte “selectOrders” metoden i case 3, 4 og 5 over hver case’s sin spesifikke metode.

Det samme ble gjort for “customer” tabellen og for “videogames” tabellen:

```
case 6:
    System.out.println("Here is all currently available games:");
    databaseConnection.selectGame();

    System.out.println(" ");

    System.out.println("What games do you want to add?: ");
    databaseConnection.createGame();
    break;
case 7:
    System.out.println("Here is all currently available games:");
    databaseConnection.selectGame();

    System.out.println(" ");

    System.out.println("What games do you want to modify?: ");
    databaseConnection.updateGame();
    break;
case 8:
    System.out.println("Here is all currently available games:");
    databaseConnection.selectGame();

    System.out.println(" ");

    System.out.println("What games do you want to return?: ");
    databaseConnection.deleteGame();
```

Figurbilde 118: Kalte “selectGame” metoden i case 6, 7 og 8 over hver case's spesifikke metode.

```
case 9:
    System.out.println("Here's the current customer info:");
    databaseConnection.selectCustomer();

    System.out.println(" ");

    System.out.println("Registered as customer");
    databaseConnection.createCustomer();
    break;
case 10:
    System.out.println("Here's the current customer info:");
    databaseConnection.selectCustomer();

    System.out.println(" ");

    System.out.println("Customer info updated: ");
    databaseConnection.updateCustomer();
    break;
case 11:
    System.out.println("Here's the current customer info:");
    databaseConnection.selectCustomer();

    System.out.println(" ");

    System.out.println("Customer removed from database");
    databaseConnection.deleteCustomer();
```

Figurbilde 119: Kalte “selectCustomer” metoden i case 9, 10 og 11, over hver case’s spesifikke metode

Andre Fase: Implementering av OOP:

Lagde en ny klasse kalt for “CustomerDAO”, som arver data fra ”DatabaseConnection”, spesifikt “dbURL” “username” og “password”. Jeg har da flytta alle CRUD operasjoner for “customer” tabellen til “CustomerDAO”. (Fungere på samme måte som før så er ikke sikker hva fordelene med dette er annet enn separasjon og responsibilitet):

```
1  import java.sql.DriverManager;
2  import java.sql.PreparedStatement;
3  import java.sql.ResultSet;
4  import java.sql.Statement;
5
6  public class CustomerDAO extends DatabaseConnection{ 2 usages new *
7      WriteData writeData = new WriteData(); 3 usages
8
9      void selectCustomer(){ 4 usages new *
10         try{
11             connection = DriverManager.getConnection(dbURL, username, password);
12
13             Statement statement = connection.createStatement();
14
15             ResultSet resultSet = statement.executeQuery( sql: "SELECT * FROM customer");
16
17             while (resultSet.next()) {
18                 System.out.println("CustomerID: " + resultSet.getInt( columnLabel: "customerID") + "\n"
19                     + "First Name: " + resultSet.getString( columnLabel: "customerFirstName") + "\n"
20                     + "Last Name: " + resultSet.getString( columnLabel: "customerLastName") + "\n"
21                     + "Email: " + resultSet.getString( columnLabel: "customerEmail"));
22             }
23             resultSet.close();
24             statement.close();
25             connection.close();
26         }catch(Exception e){
27             System.out.println("Failed to load data from table 'customer' ");
28         }
```

Figurbilde 120: “CustomerDAO” klassen med full CRUD funksjonalitet

Gjorde det samme for “OrdersDAO”

```
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;

public class OrdersDAO extends DatabaseConnection{ 2 usages new *
    WriteData writeData = new WriteData(); 3 usages

    void selectOrders(){ 4 usages new *
        try{
            connection = DriverManager.getConnection(dbURL, username, password);

            Statement statement = connection.createStatement();

            ResultSet resultSet = statement.executeQuery( sql: "SELECT * FROM orders");

            while (resultSet.next()){
                System.out.println("OrderID: " + resultSet.getInt( columnLabel: "orderID") + " \n"
                    + "First name: " + resultSet.getString( columnLabel: "customerFirstName") + " \n"
                    + "Last Name: " + resultSet.getString( columnLabel: "customerLastName") + " \n"
                    + "Game name: " + resultSet.getString( columnLabel: "gameName") + " \n"
                    + "GameID: " + resultSet.getInt( columnLabel: "gameID") + " \n"
                    + "Purchase Date: " + resultSet.getString( columnLabel: "purchaseDate") + " \n"
                    + "Returned: " + resultSet.getBoolean( columnLabel: "returned"));
            }
        }
    }
}
```

Figurbilde 121: “OrdersDAO” klassen som arver data fra “DatabaseConnection” klassen, med full CRUD funksjonalitet

Samme prosess ble også gjort for “VideoGamesDAO”:

```
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;

public class VideoGameDAO extends DatabaseConnection{ 2 usages new *
    WriteData writeData = new WriteData(); 3 usages

    void selectGame(){ 4 usages new *
        try{
            connection = DriverManager.getConnection(dbURL, username, password);

            Statement statement = connection.createStatement();

            ResultSet resultSet = statement.executeQuery( sql: "SELECT * FROM videogames");

            while (resultSet.next()){
                System.out.println("GameID: " + resultSet.getInt( columnLabel: "gameID") + " \n"
                    + "Game name: " + resultSet.getString( columnLabel: "gameName") + " \n"
                    + "Game Description: " + resultSet.getString( columnLabel: "gameDesc") + " \n"
                    + "Game Genre: " + resultSet.getString( columnLabel: "gameGenre") + " \n" + "Player mode: " + r
                    + "Price: " + resultSet.getInt( columnLabel: "price") + " \n" + "PEGI rating: " + resultSet.g
                    + "In stock: " + resultSet.getInt( columnLabel: "quantity"));
            }
        }

        resultSet.close();
        statement.close();
    }
}
```

Figurbilde 122: “VideoGamesDAO” klassen som arver data fra “DatabaseConnection” klassen, med full CRUD funksjonalitet

Grunnen til at jeg lagde “VideoGamesDAO”, “CustomerDAO” og “OrdersDAO” er fordi jeg oppdaget at “DatabaseConnection” klassen hadde nesten 300 linjer med kode, der den gjorde alt av CRUD for alle tre tabellene. I begynnelsen gikk det greit, da jeg bare hadde “READ” metodene for hver tabell, men da de andre operasjonene ble lagt til, gjorde det jobben vanskeligere å lese kode og skalere det på et senere tidspunkt. Etter å lest litt rundt på nettet, så fant jeg ut at klassen som skal koble til databasen skal bare håndtere koblingen, ikke alt av CRUD. Det gjør det også naturligvis mulig å bruke “inheritance” i prosjektet, der “VideoGamesDAO”, “CustomerDAO” og “OrdersDAO”, kan inherite dataen “dbURL”, “password”, “username” og “connection” for CRUD metodene deres

Denne endringen bidro også med implementering av “encapsulation” i programmet. “VideoGamesDAO”, “CustomerDAO” og “OrdersDAO” krever tilgang til “dbURL”, “username” og “password”, og da var en nødt til å omgjøre accessen av feltene til public. Problemet er da at feltene blir tilgjengelig for alle fremtidige klasser når det lages en ny instanse av “DatabaseConnection” klassen, noe som ikke er helt ønskelig. Da var løsningen å fortsatt beholde private keyworden, men heller å bruke “encapsulation” for hvert felt. Dette gjør at feltene kan fortsatt være private, noe som forhindrer direkte aksess til feltene, og heller at klassen må bruke feltenes getter metoder for å ha tilgang.

```
private static String dbURL = "jdbc:mysql://localhost:3306/gamestop"; 2 usages 11 related problems
private static String username = " "; 2 usages 11 related problems
private static String password = " "; 2 usages 11 related problems

public String getDbURL() { 1 usage new *
    return dbURL;
}

public void setDbURL(String newdbURL) { no usages new *
    this.dbURL = newdbURL;
}

public String getUsername() { new *
    return username;
}

public void setUsername(String newusername) { new *
    this.username = newusername;
}

public String getPassword() { new *
    return password;
}

public void setPassword(String newpassword) { new *
    this.password = newpassword;
}
```

Figurbilde 123: La til encapsulation slik at “CustomerDAO”, “VideoGameDAO” og “OrdersDAO” har tilgang til connection “dbURL”, “username” og “password”.

Dette er da gjort for alle CRUD metodene i “CustomerDAO”, “VideoGamesDAO” og “OrdersDAO”. Skjermbildet under er et eksempel på hvordan getters ble brukt for å gi CREATE metoden til “OrdersDAO” tilgang til feltene fra figurbilde 123:

```
void createOrders(){ //usage new *
    try{
        connection = DriverManager.getConnection(databaseConnection.getDbURL(), databaseConnection.getUsername(), databaseConnection.getPassword());

        String createSQL = "INSERT into orders(orderID, customerFirstName, customerLastName, gameName, gameID, purchaseDate, returned) " +
            "VALUES (?, ?, ?, ?, ?, ?, ?)";

        PreparedStatement preparedStatement = connection.prepareStatement(createSQL);

        writeData.writeDataIntoOrders(preparedStatement);
        int rowsAffected = preparedStatement.executeUpdate();

        System.out.println(rowsAffected + " rows affected");
        preparedStatement.close();
        connection.close();
    } catch (Exception e) {
        System.out.println("Couldn't insert data to 'orders'");
    }
}
```

Figurbilde 124: Bruk av getter metodene til “DatabaseConnection” klassen for å danne connection i “createOrders” metoden